

# Technical Supplement: How the TADP-Sec Runtime Protection Mechanisms Work

Modified DF+KS, xMK-CKKS, and SAMK

Supplementary technical note for the TADP-Sec framework

## Abstract

This supplement gives a compact mathematical and workflow-level explanation of the three protected aggregation mechanisms used as TADP-Sec runtime profiles: modified Domingo–Ferrer with key switching (DF+KS), xMK-CKKS, and SAMK. The purpose is explanatory rather than to reproduce the full cryptographic proofs in the source papers. For each mechanism, the note separates the original cryptographic idea from its use inside TADP-Sec, identifies what is encrypted, who can decrypt the protected aggregate, and how one federated-learning round proceeds.

## 1 Where the mechanisms fit in TADP-Sec

TADP-Sec first closes the admitted cohort using the TADP governance process. Let  $\mathcal{A}$  be the admitted clients. Each admitted client receives a minimum runtime requirement from validated confidentiality, integrity, availability, and Business Impact inputs. The server selects one session-wide profile using the highest policy-ranked requirement:

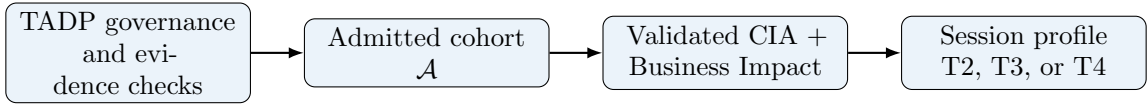
$$T_{\text{session}} = \max_{i \in \mathcal{A}}^{\text{policy}} T_i. \quad (1)$$

The selected profile is fixed for that session. T2, T3, and T4 are *alternative*, not cumulative, protected-aggregation mechanisms.

For a federated round, client  $i$  produces a local model update  $\mathbf{w}_i$ . With normalized FedAvg weight  $\alpha_i$ , the target aggregate is

$$\mathbf{w}_{\text{agg}} = \sum_{i \in \mathcal{A}} \alpha_i \mathbf{w}_i, \quad \sum_{i \in \mathcal{A}} \alpha_i = 1. \quad (2)$$

T2 performs the weighting through protected arithmetic. In the TADP-Sec T3 and T4 implementations, the contribution can be weighted before encryption and the cryptographic protocol securely sums the resulting contributions.



## 2 T2: Modified Domingo–Ferrer with Key Switching

### 2.1 Core idea

The modified DF construction used as the basis of T2 comes from Hariss, Samhat, and Chamoun [1]. Its two important ideas are: (i) replace powers of one DF secret with a vector of independent invertible secret values, and (ii) use a public key-switching matrix to reduce ciphertext dimension after homomorphic multiplication. The original paper addresses cloud computation; using it for protected FedAvg is a TADP-Sec adaptation.

Let the fixed-point encoding of a scalar value  $x$  be

$$Q_S(x) = \text{Round}(Sx), \quad (3)$$

where  $S$  is the scale. For plaintext  $a = Q_S(x) \in \mathbb{Z}_{m_0}$ , choose random shares  $a_1, \dots, a_d \in \mathbb{Z}_m$  such that

$$\sum_{j=1}^d a_j \equiv a \pmod{m_0}. \quad (4)$$

With secret vector  $\mathbf{s} = (r_1, \dots, r_d)$ , the modified DF ciphertext is

$$\mathbf{c} = (a_1 r_1, \dots, a_d r_d) \pmod{m}. \quad (5)$$

Decryption removes the secret factors and recombines the shares:

$$\text{Dec}_{\mathbf{s}}(\mathbf{c}) = \left( \sum_{j=1}^d c_j r_j^{-1} \right) \pmod{m_0}. \quad (6)$$

Addition is coordinate-wise. Multiplication creates all square and cross terms, so the ciphertext expands from  $d$  coordinates to

$$D = d + \binom{d}{2}. \quad (7)$$

The expanded ciphertext  $\mathbf{c}^* \in \mathbb{Z}_m^D$  is associated with an expanded secret  $\mathbf{s}^*$  containing terms such as  $r_j^2$  and  $r_j r_k$ . This expansion is precisely the problem addressed by key switching. The source paper shows the same phenomenon more generally: repeated homomorphic multiplication increases ciphertext dimension rapidly unless KS is applied [1].

## 2.2 Key switching without decryption

Key switching maps the expanded ciphertext to a new  $d$ -dimensional key domain while preserving the plaintext:

$$\text{Dec}_{\mathbf{s}^*}(\mathbf{c}^*) = \text{Dec}_{\mathbf{s}^0}(\mathbf{c}^0), \quad \mathbf{c}^0 = M\mathbf{c}^*. \quad (8)$$

The public matrix  $M \in \mathbb{Z}_m^{d \times D}$  is constructed so that

$$(\mathbf{s}^0)^{-1} M = m_0 \mathbf{e} + (\mathbf{s}^*)^{-1}, \quad (9)$$

for a random noise vector  $\mathbf{e}$ . Therefore,

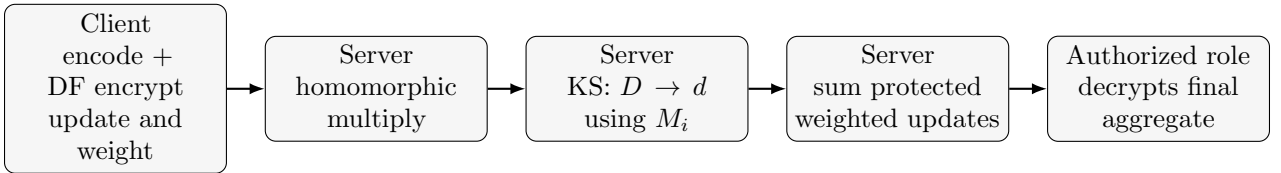
$$(\mathbf{s}^0)^{-1} \mathbf{c}^0 \pmod{m_0} = (\mathbf{s}^0)^{-1} M \mathbf{c}^* \pmod{m_0} \quad (10)$$

$$= (\mathbf{s}^*)^{-1} \mathbf{c}^* \pmod{m_0}. \quad (11)$$

The  $m_0 \mathbf{e}$  term vanishes modulo  $m_0$ . Thus KS changes the key/domain of the ciphertext without decrypting and re-encrypting the protected value. In the construction,  $M = [\mathbf{b}; A]$ , where  $A$  is sampled uniformly and  $\mathbf{b}$  hides the old extended secret under the new secret using an LWE-style noisy relation [1].

## 2.3 TADP-Sec T2 workflow

For each model coordinate, T2 protects both the local update and the FedAvg weight. If  $X_i = Q_S(w_i)$  and  $A_i = Q_S(\alpha_i)$ , homomorphic multiplication produces a protected representation of  $A_i X_i$ , which is later rescaled by  $S^2$  to approximate  $\alpha_i w_i$ .



In the TADP-Sec implementation, one client-to-aggregation switching matrix  $M_i$  is generated during session setup and reused across rounds. The *matrix setup* is one-time; the actual key-switching operation is still executed on protected values during the relevant training rounds.

**Interpretation.** T2 demonstrates protected fixed-point aggregation with dimension-reducing KS. It should be treated as a reference/proof-of-concept profile rather than as equivalent in cryptographic maturity to the RLWE-based T3 and T4 profiles. The underlying DF family is a legacy algebraic construction and the source paper itself focuses on improving resistance to known-plaintext analysis and ciphertext expansion [1].

### 3 T3: xMK-CKKS Multi-Key Aggregation

#### 3.1 Keys and encrypted updates

xMK-CKKS was proposed specifically for privacy-preserving FL [2]. CKKS represents approximate real/complex values as polynomials in an RLWE ring, here written as

$$\mathcal{R}_q = \mathbb{Z}_q[X]/(X^N + 1). \quad (12)$$

Each client  $i$  samples a secret  $s_i$  and forms a public-key component

$$b_i = -s_i a + e_i \pmod{q}, \quad (13)$$

where  $a$  is common public randomness and  $e_i$  is small RLWE noise. The clients' public-key components are combined into one aggregate public key

$$\tilde{b} = \sum_{i=1}^n b_i. \quad (14)$$

An encoded client contribution  $m_i$  is encrypted as

$$ct_i = (c_{i,0}, c_{i,1}) = (v_i \tilde{b} + m_i + e_{i,0}, v_i a + e_{i,1}) \pmod{q}. \quad (15)$$

The server adds ciphertexts directly:

$$C_{\text{sum}} = (C_0, C_1) = \sum_{i=1}^n ct_i. \quad (16)$$

#### 3.2 Collaborative decryption

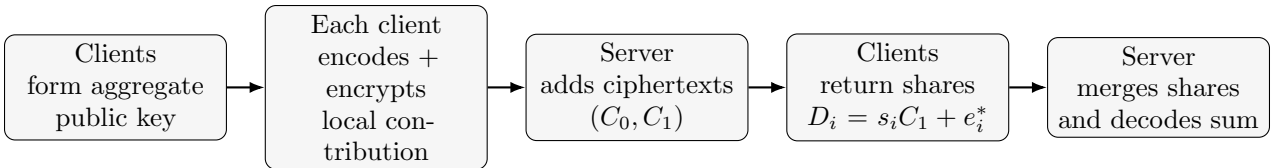
The important xMK-CKKS change is that a client's decryption share is bound to the *aggregate* ciphertext component rather than to that client's individual ciphertext. Client  $i$  returns

$$D_i = s_i C_1 + e_i^* \pmod{q}, \quad (17)$$

where the added flooding noise protects the secret share. The server merges the aggregate first component with all client shares:

$$C_0 + \sum_{i=1}^n D_i \approx \sum_{i=1}^n m_i \pmod{q}. \quad (18)$$

After CKKS decoding, the aggregate is recovered approximately. In the original xMK-CKKS FL protocol, the server obtains the aggregate plaintext but not the individual client updates [2].



**Why the individual update is not directly decrypted.** If a device supplied a share based only on its own  $c_{i,1}$ , the server could combine it with  $c_{i,0}$  and approximately recover  $m_i$ . xMK-CKKS instead uses  $C_1 = \sum_i c_{i,1}$ , so each  $D_i$  is tied to the joint aggregate. The source paper analyzes the protocol under an honest-but-curious model and considers collusion of the server with fewer than  $N - 1$  participating devices [2]. This supplement does not extend that proof to unrelated active attacks such as poisoning or Byzantine model manipulation.

**TADP-Sec use.** TADP-Sec applies xMK-CKKS only after governance closes the admitted cohort. The active key set therefore corresponds to the clients allowed to participate in that session. Packed CKKS encoding allows multiple model coordinates to share one polynomial ciphertext, which keeps the update ciphertext much smaller than encrypting every coordinate independently.

## 4 T4: SAMK Authorized-Client Aggregate Recovery

### 4.1 The BFV decryption term becomes a polynomial helper

SAMK combines BFV, Paillier encryption, and polynomial interpolation so that clients using different keys can independently recover the aggregate without collaborative client-to-client decryption [3]. Let  $n$  denote the number of clients and  $N$  the BFV polynomial degree/number of interpolation points.

Client  $i$  encrypts an integer-encoded model contribution  $m_i$  with its BFV key, obtaining

$$ct_i = (c_{i,0}, c_{i,1}). \quad (19)$$

Standard BFV decryption uses the secret-key product  $s_i c_{i,1}$ . SAMK names this polynomial

$$f_i(x) = s_i c_{i,1}(x). \quad (20)$$

For the uploaders  $H$ , the server can add the first BFV components and the helper polynomials conceptually:

$$CT_{\text{agg}} = \sum_{i \in H} c_{i,0}, \quad F(x) = \sum_{i \in H} f_i(x). \quad (21)$$

If an authorized client can reconstruct  $F(x)$ , the aggregate follows from the usual BFV decryption relation:

$$m_{\text{agg}} = \left[ \left\lfloor \frac{t}{q} [CT_{\text{agg}} + F(x)]_q \right\rfloor \right]_t. \quad (22)$$

The challenge is therefore to let an authorized uploader reconstruct  $F(x)$  while preventing the server from learning enough information to do the same.

### 4.2 Protecting interpolation points with Paillier

Choose  $N$  distinct public points  $X = \{x_0, \dots, x_{N-1}\}$ . A polynomial of degree at most  $N-1$  is uniquely determined by these  $N$  points. Each client evaluates  $f_i(x_t)$  at all points.

- For  $t = 0, \dots, N-2$ , client  $i$  encrypts  $f_i(x_t)$  under the *server's* Paillier public key.
- For the final point  $x_{N-1}$ , client  $i$  encrypts  $f_i(x_{N-1})$  separately under each intended client's Paillier public key.

Paillier is additively homomorphic: multiplying ciphertexts corresponds to adding their plaintexts. Hence the server can obtain encrypted values of

$$F(x_t) = \sum_{i \in H} f_i(x_t) \quad (23)$$

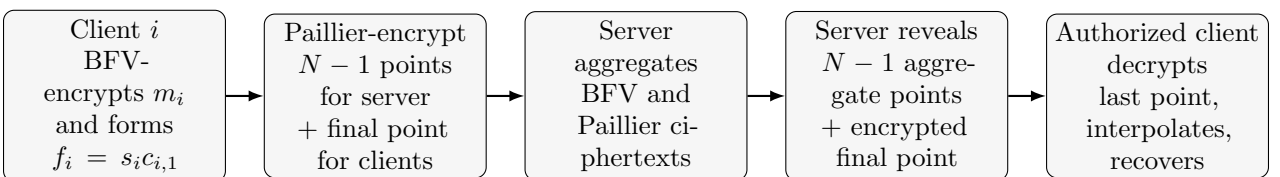
without learning the individual  $f_i(x_t)$  values. The server decrypts only the first  $N-1$  aggregate points. For an authorized client  $j$ , the server sends

$$\{CT_{\text{agg}}, F(x_0), \dots, F(x_{N-2}), \text{Enc}_{pk_j}(F(x_{N-1}))\}. \quad (24)$$

Client  $j$  decrypts the last point with its own Paillier secret key. It now has all  $N$  points and reconstructs

$$F(x) = \sum_{t=0}^{N-1} F(x_t) L_t(x), \quad L_t(x) = \prod_{k \neq t} \frac{x - x_k}{x_t - x_k}, \quad (25)$$

then uses  $F(x)$  with  $CT_{\text{agg}}$  to recover  $m_{\text{agg}}$ . The server has only  $N-1$  plaintext points and therefore cannot reconstruct the degree- $(N-1)$  helper polynomial from those points alone [3].



### 4.3 Dropout behavior

SAMK naturally separates clients who uploaded from those who did not. Let  $H \subseteq \mathcal{A}$  be the set of successful uploaders.

- **Pre-upload dropout:** a non-uploader is excluded from the survivor aggregate. Its missing model update is *not reconstructed*. The server does not provide that client with the client-specific final interpolation point needed to recover the result.
- **Post-upload temporary unavailability:** once a client’s protected upload is included in  $H$ , that client may reconnect later and use its client-specific recovery material to obtain the same aggregate. This is delayed recovery, not reconstruction of a missing update.

The SAMK paper explicitly designs for clients going offline before or after upload and for later recovery by a client that had already uploaded [3].

**TADP-Sec use.** In T4, the server performs protected aggregation and helper processing but does not obtain the plaintext aggregate. Authorized successful uploaders recover it independently. In the single-process experimental harness, one validated recipient path can be mirrored locally only to advance the FL model; this represents authorized-client recovery and should not be interpreted as server decryption.

## 5 Side-by-side interpretation

Table 1: Operational difference among the three protected profiles.

| Profile | Core<br>nism                             | mecha-<br>nism | How aggregation stays<br>protected                                                                                                             | Who obtains aggregate<br>plaintext?                                                                            | Main operational conse-<br>quence                                                                                       |
|---------|------------------------------------------|----------------|------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| T2      | Modified DF +<br>KS                      |                | Fixed-point ciphertext arithmetic; multiplication expands the ciphertext and KS maps it back to the aggregation-key dimension.                 | A designated authorized/classified decryption role, not the aggregation server.                                | Per-coordinate DF objects and KS matrices create storage/communication overhead.                                        |
| T3      | xMK-CKKS                                 |                | Clients encrypt under one aggregate public key; the server adds ciphertexts; clients return aggregate-bound decryption shares.                 | The server can recover the aggregate after receiving the required shares; individual updates remain encrypted. | Efficient polynomial packing, but collaborative decryption adds a share exchange.                                       |
| T4      | SAMK (BFV +<br>Paillier + interpolation) |                | BFV protects updates; Paillier protects helper-polynomial points; each authorized uploader reconstructs the missing helper polynomial locally. | Authorized successful uploaders; the server does not receive the plaintext aggregate.                          | Stronger recovery separation and dropout support, at the cost of substantial helper ciphertext traffic and computation. |

## 6 TADP-Sec experimental instantiation

Table 2 records the parameterization used by the current TADP-Sec reference implementation. These values document the evaluated prototype; they are not an independent cryptographic certification.

Table 2: Reference implementation parameters used in TADP-Sec.

| Profile | Primitive        | Evaluated parameterization                                                                                                                                                          |
|---------|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| T2      | Modified DF + KS | $d = 25$ , security parameter $\lambda = 10$ , $m' = 2^{80}$ , $m = (m')^\lambda = 2^{800}$ , fixed-point scale $S = 10^6$ . T2 is treated as a proof-of-concept legacy-HE profile. |
| T3      | xMK-CKKS         | RLWE polynomial degree $N = 2048$ , 54-bit total coefficient modulus, CKKS scale $2^{40}$ .                                                                                         |
| T4      | SAMK             | BFV with $N = 2048$ , 54-bit ciphertext-modulus setting, plaintext modulus $t = 16,777,213$ , fixed-point scale $10^6$ ; Paillier modulus approximately 3072 bits.                  |

The T3 and T4 RLWE components are configured to target approximately 128-bit classical security under the parameter guidance used in the main manuscript. T2 is deliberately not assigned the same claim because modified DF is a different, legacy construction with different security assumptions.

## 7 What these mechanisms do – and do not do

The three mechanisms protect the confidentiality and handling of model-update aggregation in different ways. They do not, by themselves, establish that a client update is statistically benign or that a client trained honestly. In TADP-Sec, signature checks, freshness/anti-replay controls, participation control, and audit logging are separate protocol controls. Likewise, poisoning, backdoors, Byzantine optimization, and network-level denial-of-service require defenses beyond the protected-aggregation primitive.

The important distinction is therefore:

Governance decides who may join; runtime profiles decide how admitted contributions are protected.

T2 demonstrates protected multiplication with key switching, T3 demonstrates packed multi-key aggregation with collaborative decryption, and T4 demonstrates multi-key secure aggregation with authorized-client recovery and dropout-aware availability.

## Source note

The mathematical descriptions above intentionally summarize only the parts needed to understand the TADP-Sec workflows. For formal security proofs, parameter constraints, cryptanalysis, and full algorithms, the original papers should be consulted directly. TADP-Sec-specific FedAvg weighting, governance admission, session-level profile selection, and the experimental orchestration described here are integration choices of the TADP-Sec framework rather than claims that those details appear in every source paper.

## References

- [1] K. Hariss, A. E. Samhat, and M. Chamoun, “An Efficient FHE Scheme to Secure Cloud Computing,” in *Proc. 16th International Joint Conference on e-Business and Telecommunications (ICETE/SECRYPT)*, 2019, pp. 341–349, doi: 10.5220/0007788803410349.
- [2] J. Ma, S.-A. Naas, S. Sigg, and X. Lyu, “Privacy-Preserving Federated Learning Based on Multi-Key Homomorphic Encryption,” arXiv:2104.06824, 2021.
- [3] C. Guo, L. Li, X. Liu, K.-K. R. Choo, and Y. Liu, “SAMK: Secure Aggregation for Federated Learning Under Multiple Keys With Low Communication Rounds,” *IEEE Transactions on Dependable and Secure Computing*, vol. 22, no. 6, 2025, doi: 10.1109/TDSC.2025.3602908.